

— CCAF CERTIFICATION SERIES

Introduction to Agent Skills

Anthropic 官方认证课程完整笔记
6节课精华整理 + 考试考点映射

organized by Syneira
2026.06

Skills 是什么

Skills 是可复用的 markdown 指令文件，Claude Code 根据任务自动匹配并加载。写一次，之后每次遇到匹配的任务，Claude 自动应用。

"Skills are reusable markdown instruction files. You teach Claude once, and it applies that knowledge automatically whenever a matching task comes up."

核心概念

每个 Skill 是一个文件夹，内含一个 SKILL.md 文件。文件分两部分：frontmatter（元数据）和具体指令。Claude 用 description 字段进行语义匹配，当你的请求和描述含义重叠时，Skill 被激活。

"Each skill lives in its own named folder containing a SKILL.md file. The file has two parts: YAML frontmatter with metadata, and the instruction body below."

```
---
name: pr-review
description: Reviews pull requests for code quality.
              Use when reviewing PRs or checking code changes.
---

When writing a PR description:
1. Run `git diff main...HEAD` to see all changes
2. Write a description following this format:
   ## What / ## Why / ## Changes
```

存放位置

位置	路径	共享范围
个人	~/.claude/skills/	仅自己，跨所有项目生效
项目	.claude/skills/	通过 Git 共享给团队

Windows 下个人路径为 C:/Users/<user>/.claude/skills。项目 Skills 随代码提交到版本控制，团队成员 clone 后自动获得。

加载机制

"At startup, Claude only loads the name and description of each skill. The full content is loaded on-demand when a request matches."

- 启动时只加载每个 Skill 的 **name + description**，不占满 context window
- 当请求匹配时，Claude 提示确认后才加载完整 SKILL.md 内容

- 按需加载意味着你可以有很多 Skills 而不影响日常对话

优先级

优先级（考试高频）：Enterprise > Personal > Project > Plugins

同名冲突时，高优先级覆盖低优先级。建议使用有区分度的命名避免冲突，如 `frontend-review` 而非 `review`。

创建与测试

从零创建一个 Skill 只需两步：建目录、写 SKILL.md。

"Creating a skill is straightforward: make a folder, add a SKILL.md, and restart Claude Code."

创建步骤

1. 在 skills 文件夹下创建以 skill 名称命名的目录：`mkdir -p ~/.claude/skills/pr-description`
2. 在该目录下创建 `SKILL.md` 文件，写入 frontmatter + 指令
3. 重启 Claude Code，新 Skill 在下次启动时可用
4. 用匹配的请求测试（如 "write a PR description for my changes"），观察 Skill 是否被加载

Frontmatter 结构

字段	必填	说明
name	✔	标识符，小写字母 + 数字 + 连字符，最长 64 字符，应与目录名一致
description	✔	Claude 用来匹配请求的依据，最长 1024 字符。这是最重要的字段
allowed-tools		限制 Skill 激活时可用的工具列表
model		指定使用哪个 Claude 模型

更新与删除

更新：直接编辑 `SKILL.md` 。删除：删除整个 Skill 目录。所有更改需**重启 Claude Code** 后生效。

"To update a skill, edit the SKILL.md file directly. To remove it, delete the entire skill folder. Changes take effect after restarting Claude Code."

高级配置与多文件结构

通过 `allowed-tools` 限制工具访问，通过多文件结构实现 `progressive disclosure`，在保持 `context` 高效的同时支持复杂场景。

"Use `allowed-tools` to restrict tool access, and multi-file structures for `progressive disclosure`, keeping the context window efficient while supporting complex workflows."

`allowed-tools`：工具限制

有时你希望 Skill 只能读取文件而不能修改，适用于安全审计、只读分析等场景：

```
---
name: codebase-onboarding
description: Helps new developers understand the codebase.
allowed-tools: Read, Grep, Glob, Bash
model: sonnet
---
```

当 Skill 激活时，Claude 只能使用声明的工具，其他工具被限制。如果省略 `allowed-tools`，则不做任何限制，沿用正常权限模型。

Progressive Disclosure（渐进式加载）

"Don't put everything in a single 2000-line SKILL.md. Use `progressive disclosure`: keep the core instructions short, and put detailed references in subdirectories that Claude reads on demand."

将所有内容塞进一个 2000 行的 SKILL.md 会占用大量 `context window` 且难以维护。更好的做法：

- SKILL.md 控制在 **500 行以内**，只放核心指令
- 详细参考放在 `references/` 子目录，Claude 按需读取
- 脚本放在 `scripts/` 子目录，执行时不加载源码到 `context`，**只有输出消耗 token**
- 静态资源放在 `assets/` 子目录

推荐目录结构

```
my-skill/
├── SKILL.md           # 核心指令 (< 500 行)
├── scripts/           # 可执行脚本
├── references/         # 参考文档
└── assets/            # 模板、图片等
```

在 SKILL.md 中用清晰的指令告诉 Claude 何时读取子文件：

When the user asks about system architecture, read `references/architecture-guide.md`.

写好 Description

"The description is how Claude decides whether to load a skill. A good description answers: what does this skill do, and when should it be used?"

Description 是 Claude 匹配请求的唯一依据。一个好的 description 应该回答两个问题：

1. 这个 Skill 做什么？
2. 什么时候应该使用它？

如果 Skill 没有按预期触发，先检查 description 是否包含了用户实际会使用的关键词和表述方式。

Skills 与其他功能的对比

Claude Code 有多种自定义方式。选错了会增加不必要的复杂度，选对了才能各司其职。

"Claude Code offers several customization mechanisms. Choosing the right one avoids unnecessary complexity."

核心对比表

功能	加载时机	触发方式	隔离性	适用场景
CLAUDE.md	每次对话都加载	自动 (always-on)	无	项目级标准，始终生效的规则
Skills	按需匹配加载	自动 (request-driven)	无	特定任务的专业知识
Slash Commands	不自动加载	手动输入命令	无	用户主动调用的操作
Hooks	事件触发	自动 (event-driven)	无	文件保存/工具调用时的自动操作
Subagents	独立上下文	委派任务	隔离	需要独立执行环境的委派工作

选择判断框架

- 需要**始终生效**（如 "永远用 TypeScript strict mode"）→ CLAUDE.md
- 需要**按任务自动匹配**（如 "PR review 用这套 checklist"）→ Skills
- 需要**手动调用**（如 "/deploy 触发部署"）→ Slash Commands
- 需要**确定性合规**（如 "每次保存文件都跑 linter"）→ Hooks
- 需要**隔离执行**（如 "交给独立 agent 做安全审计"）→ Subagents

CLAUDE.md vs Skills 的边界

"CLAUDE.md is always-on: it loads into every conversation. Skills are on-demand: they load only when the task matches. Put universal rules in CLAUDE.md, task-specific expertise in Skills."

CLAUDE.md 适合放：项目通用的编码标准、"永远不要修改数据库 schema" 之类的硬约束、框架偏好。

Skills 适合放：PR review checklist、commit message 格式、文档模板、特定框架的 debug 流程。区别在于：**CLAUDE.md** 的内容每次都占 context，**Skills** 只在需要时加载。

共享与分发

一个人用好的 Skill 可以帮团队标准化 workflow，前提是用对分发方式。

"Skills you build for yourself can standardize workflows for your whole team, if you choose the right distribution method."

三种分发方式

方式	适用场景	优先级	机制
项目 <code>.claude/skills/</code>	团队标准	Project	Git commit, clone 即获取
Plugins	跨项目社区分发	最低	通过 marketplace 安装
Enterprise Managed	组织强制标准	最高	管理员统一部署，不可覆盖

Enterprise 管理设置支持 `strictKnownMarketplaces` 字段，限制 plugin 只能从指定来源安装（如公司内部 GitHub repo 或 npm scope）。

Subagent 与 Skills 的关系

"Subagents do NOT automatically inherit skills from the parent session. Built-in agents (Explorer, Plan, Verify) cannot access skills at all. Custom subagents must explicitly declare skills in their frontmatter."

重要陷阱（考试常考）：

- ✗ Subagent **不自动继承**当前会话的 Skills
- ✗ 内置 agent（Explorer、Plan、Verify）**完全无法访问** Skills
- ✓ 只有自定义 subagent 在其 frontmatter 中**显式声明** `skills` 字段才能使用

创建自定义 subagent 时，在 `.claude/agents/` 目录下定义 agent 文件，用 `/agents` 命令交互式创建：

```
---
name: frontend-security-accessibility-reviewer
description: "Reviews frontend code for accessibility..."
tools: Bash, Glob, Grep, Read, WebFetch, WebSearch
model: sonnet
skills: accessibility-audit, performance-check
---
```

Skills 在 subagent 启动时加载（不是按需），且列出的 Skills 必须存在于 `.claude/skills/` 目录中。

排错指南

Skill 不工作时，问题通常可以归为几个可预测的类别。

"When a skill doesn't work, the issue usually falls into a few predictable categories: not triggering, not loading, wrong skill selected, being overridden, or runtime errors."

常见问题速查

问题	常见原因	解决方法
Skill 不触发	description 和用户请求语义不匹配	加入用户实际会说的关键词和表述方式
Skill 不加载	文件结构错误	SKILL.md 必须在命名目录内（不是直接放在 skills/ 根目录），文件名大小写正确
用错了 Skill	多个 Skill 的 description 太相似	让 description 更具体、有区分度
被覆盖	优先级冲突（同名 Skill 存在于更高优先级位置）	重命名或检查 Enterprise > Personal > Project > Plugins 层级
运行时错误	依赖缺失、权限不足、路径问题	确保 <code>chmod +x</code> 给脚本、路径用前斜杠、安装所需依赖
Plugin Skills 不出现	缓存问题或 plugin 结构错误	清除缓存，重启 Claude Code，重新安装 plugin

调试工具

- `claude --debug`：启动时查看 Skill 加载日志，定位加载错误
- **Agent skills validator**：第三方工具，检查 Skill 目录结构和 frontmatter 格式是否正确
- 询问 Claude "what skills are available"：验证 Skill 是否成功注册

排错优先级

1. 先用 validator 检查结构问题
2. 再用 `claude --debug` 查看加载日志
3. 如果加载正常但不触发，调整 description 的关键词
4. 如果触发但行为不对，检查指令内容和 allowed-tools 配置

考试映射：Task Statement 3.2

课程覆盖了 CCAF 考试 Task Statement 3.2 的大部分考点，但有几个重要考点课程没有深入讲解。

"Task Statement 3.2: Customize the behavior of Claude Code by writing skills, slash commands, and hooks."

课程已覆盖的考点

考点	对应课程内容
覆盖 Skills 存放位置	Lesson 1-2: personal (<code>~/.claude/skills/</code>) vs project (<code>.claude/skills/</code>)
覆盖 SKILL.md frontmatter	Lesson 2-3: name, description, allowed-tools, model 字段
覆盖 Skills vs 其他功能	Lesson 4: Skills vs CLAUDE.md vs Slash Commands vs Hooks vs Subagents 详细对比
覆盖 context: fork 概念	Lesson 5: 通过 subagent 隔离执行的概念（但未深入具体配置）

课程未讲但考试会考的

以下考点需要额外学习：

考点	说明
未覆盖 context: fork 具体配置	考纲 3.2 明确提到 context: fork 作为 frontmatter 选项，课程仅通过 subagent 概念间接涉及
未覆盖 argument-hint frontmatter	用于提示用户输入参数的 frontmatter 字段
未覆盖 .claude/commands/slash commands	考纲 3.2 涵盖，课程仅在对比时提及，未深入讲解如何创建
未覆盖 Skills 在 CI/CD 中的应用	自动化流水线中 Skills 的使用场景
未覆盖 Skills vs CLAUDE.md 判断题	考试中频繁出现的场景判断：给定需求选择正确的自定义方式

典型考题示例

你的团队需要一个代码审查流程，在 Skill 执行时产生大量 verbose 输出。如何防止这些输出污染主对话上下文？

答案：使用 `context: fork` 在隔离的 sub-agent 中运行 Skill。

备考建议

1. 课程内容作为基础，重点掌握**功能对比表**和**优先级顺序**
2. 额外学习 `context: fork` 和 `argument-hint` 的具体用法
3. 练习场景判断题：给定一个需求，快速判断应该用 CLAUDE.md、Skills、Slash Commands、Hooks 还是 Subagents
4. 理解 Subagent 与 Skills 的关系：不自动继承、需显式声明、内置 agent 完全无法访问

organized by Syneira

基于 Anthropic CCAF 认证课程 "Introduction to Agent Skills" (6 lessons) 整理

课程完成度 100% • 2026.06